MÓDULO 1 · ARQUITECTURA ASÍNCRONA

Orquestador Concurrente de Modelos

Diego Lalanda

Python 3.12 · ejercicio opcional



Enunciado

Simular la lógica de un backend que debe consultar múltiples fuentes de IA de forma eficiente.

1. **Configuración inicial.** Entorno con Python 3.12+.
2. **Simulación de llamadas.** Tres corrutinas `async def` (`gpt_4_call`, `claude_3_call`, `local_llama_call`) con `await asyncio.sleep()` y distintas latencias.
3. **Orquestación.** Disparar las tres a la vez con `asyncio.gather`.
4. **Resiliencia.** `asyncio.timeout` de 2 segundos sobre la ejecución total. Si alguna tarda más, capturar la excepción, mostrar error y no detener el programa.
5. **Control de flujo.** `asyncio.Semaphore`: 10 simulaciones, solo 2 a la vez.

Criterios: no usar `time.sleep`; en los logs las tareas inician casi al mismo tiempo; manejar `TimeoutError`.

Resolución

```
"""
Ejercicio: Orquestador Concurrente de Modelos
Curso AI Engineer — Coderhouse (Módulo 1 · Arquitectura asíncrona)

Simula un backend que consulta varias fuentes de IA de forma concurrente,
con timeout de resiliencia y semáforo para control de flujo (rate limit).
"""

from __future__ import annotations

import asyncio
import logging
import sys
import time
```

```python
from typing import Any

# time.perf_counter se usa SOLO para medir duración. Nunca time.sleep.

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

logging.basicConfig(
    level=logging.INFO,
    stream=sys.stdout,
    format="%(asctime)s.%(msecs)03d | %(levelname)-8s | %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("orquestador")

TIMEOUT_TOTAL_S = 2.0
MAX_CONCURRENTES = 2
TOTAL_SIMULACIONES = 10

# Latencias simuladas (red). Llama supera el timeout total de 2s a propósito.
GPT4_LATENCY_S = 0.6
CLAUDE_LATENCY_S = 1.1
LLAMA_LATENCY_S = 2.8


def _elapsed(inicio: float) -> str:
    return f"{time.perf_counter() - inicio:6.3f}s"


# ---------------------------------------------------------------------------
# 1. Simulación de llamadas a distintos modelos (I/O-bound, no bloqueante)
# ---------------------------------------------------------------------------
async def gpt_4_call(prompt: str, inicio: float) -> dict[str, Any]:
    logger.info("GPT-4          | INICIO  | t=%s | prompt=%r", _elapsed(inicio), prompt)
    await asyncio.sleep(GPT4_LATENCY_S)
    logger.info("GPT-4          | FIN     | t=%s", _elapsed(inicio))
    return {
        "modelo": "gpt-4",
        "ok": True,
        "latencia_s": GPT4_LATENCY_S,
        "respuesta": f"[gpt-4] {prompt}",
    }


async def claude_3_call(prompt: str, inicio: float) -> dict[str, Any]:
    logger.info("Claude 3       | INICIO  | t=%s | prompt=%r", _elapsed(inicio), prompt)
    await asyncio.sleep(CLAUDE_LATENCY_S)
    logger.info("Claude 3       | FIN     | t=%s", _elapsed(inicio))
    return {
        "modelo": "claude-3",
        "ok": True,
        "latencia_s": CLAUDE_LATENCY_S,
        "respuesta": f"[claude-3] {prompt}",
    }


async def local_llama_call(prompt: str, inicio: float) -> dict[str, Any]:
    logger.info("Local Llama    | INICIO  | t=%s | prompt=%r", _elapsed(inicio), prompt)
    await asyncio.sleep(LLAMA_LATENCY_S)
    logger.info("Local Llama    | FIN     | t=%s", _elapsed(inicio))
    return {
        "modelo": "local-llama",
        "ok": True,
        "latencia_s": LLAMA_LATENCY_S,
        "respuesta": f"[local-llama] {prompt}",
    }


# ---------------------------------------------------------------------------
# 2 y 3. Orquestación concurrente + timeout sobre la ejecución TOTAL
# ---------------------------------------------------------------------------
async def orquestar_modelos(prompt: str) -> list[Any]:
    """Dispara las tres llamadas a la vez con asyncio.gather.
```

```
    asyncio.timeout envuelve la ejecución total (no cada llamada).
    Si alguna tarda de más y el lote supera 2s, se captura TimeoutError
    y el programa sigue (no se detiene).
    """
    inicio = time.perf_counter()
    logger.info("Orquestación   | gather de 3 modelos | timeout total=%.1fs", TIMEOUT_TOTAL_S)

    try:
        async with asyncio.timeout(TIMEOUT_TOTAL_S):
            resultados = await asyncio.gather(
                gpt_4_call(prompt, inicio),
                claude_3_call(prompt, inicio),
                local_llama_call(prompt, inicio),
                return_exceptions=True,
            )
    except TimeoutError:
        logger.error(
            "TimeoutError   | la ejecución total superó %.1fs. "
            "Se aplica fallback y el programa continúa.",
            TIMEOUT_TOTAL_S,
        )
        return []

    logger.info("Orquestación   | completada en %s", _elapsed(inicio))
    return resultados


# ---------------------------------------------------------------------------
# 4. Control de flujo: 10 simulaciones, máximo 2 concurrentes
# ---------------------------------------------------------------------------
async def simulacion_llamada(
    nro: int,
    sem: asyncio.Semaphore,
    activas: dict[str, int],
    inicio: float,
) -> dict[str, Any]:
    """Una simulación protegida por semáforo (máx. 2 a la vez)."""
    async with sem:
        activas["n"] += 1
        logger.info(
            "Simulación %02d  | INICIO  | t=%s | concurrentes=%d/%d",
            nro,
            _elapsed(inicio),
            activas["n"],
            MAX_CONCURRENTES,
        )
        await asyncio.sleep(0.4)
        activas["n"] -= 1
        logger.info(
            "Simulación %02d  | FIN     | t=%s | concurrentes=%d/%d",
            nro,
            _elapsed(inicio),
            activas["n"],
            MAX_CONCURRENTES,
        )
        return {"id": nro, "ok": True}


async def orquestar_con_semaforo() -> list[Any]:
    """Encola 10 llamadas; el semáforo deja pasar solo 2 al mismo tiempo."""
    sem = asyncio.Semaphore(MAX_CONCURRENTES)
    activas = {"n": 0}
    inicio = time.perf_counter()

    logger.info(
        "Semáforo       | disparando %d simulaciones | max concurrentes=%d",
        TOTAL_SIMULACIONES,
        MAX_CONCURRENTES,
    )

    tareas = [
        simulacion_llamada(i, sem, activas, inicio)
```

```
        for i in range(1, TOTAL_SIMULACIONES + 1)
    ]
    resultados = await asyncio.gather(*tareas, return_exceptions=True)

    logger.info(
        "Semáforo       | %d/%d OK en %s",
        sum(1 for r in resultados if isinstance(r, dict) and r.get("ok")),
        TOTAL_SIMULACIONES,
        _elapsed(inicio),
    )
    return resultados


async def main() -> None:
    prompt = "Resumí el patrón gather + timeout + semáforo."

    logger.info("=== Parte A: 3 modelos en paralelo + timeout total de 2s ===")
    resultados_modelos = await orquestar_modelos(prompt)
    if resultados_modelos:
        for item in resultados_modelos:
            logger.info("Resultado      | %s", item)
    else:
        logger.info("Resultado      | lote vacío por timeout; el flujo no se detuvo.")

    logger.info("=== Parte B: 10 simulaciones con Semaphore(2) ===")
    await orquestar_con_semaforo()

    logger.info("Ejercicio finalizado sin bloquear el event loop.")


if __name__ == "__main__":
    asyncio.run(main())
```

Salida de la consola

```
00:43:56.781 | INFO     | === Parte A: 3 modelos en paralelo + timeout total de 2s ===
00:43:56.781 | INFO     | Orquestación   | gather de 3 modelos | timeout total=2.0s
00:43:56.782 | INFO     | GPT-4          | INICIO  | t= 0.000s | prompt='Resumí el patrón gather + timeout +
semáforo.'
00:43:56.782 | INFO     | Claude 3       | INICIO  | t= 0.000s | prompt='Resumí el patrón gather + timeout +
semáforo.'
00:43:56.782 | INFO     | Local Llama    | INICIO  | t= 0.000s | prompt='Resumí el patrón gather + timeout +
semáforo.'
00:43:57.384 | INFO     | GPT-4          | FIN     | t= 0.602s
00:43:57.883 | INFO     | Claude 3       | FIN     | t= 1.101s
00:43:58.777 | ERROR    | TimeoutError   | la ejecución total superó 2.0s. Se aplica fallback y el programa
continúa.
00:43:58.777 | INFO     | Resultado      | lote vacío por timeout; el flujo no se detuvo.
00:43:58.777 | INFO     | === Parte B: 10 simulaciones con Semaphore(2) ===
00:43:58.777 | INFO     | Semáforo       | disparando 10 simulaciones | max concurrentes=2
00:43:58.778 | INFO     | Simulación 01  | INICIO  | t= 0.000s | concurrentes=1/2
00:43:58.778 | INFO     | Simulación 02  | INICIO  | t= 0.000s | concurrentes=2/2
00:43:59.183 | INFO     | Simulación 01  | FIN     | t= 0.405s | concurrentes=1/2
00:43:59.183 | INFO     | Simulación 02  | FIN     | t= 0.406s | concurrentes=0/2
00:43:59.183 | INFO     | Simulación 03  | INICIO  | t= 0.406s | concurrentes=1/2
00:43:59.183 | INFO     | Simulación 04  | INICIO  | t= 0.406s | concurrentes=2/2
00:43:59.585 | INFO     | Simulación 03  | FIN     | t= 0.807s | concurrentes=1/2
00:43:59.585 | INFO     | Simulación 04  | FIN     | t= 0.807s | concurrentes=0/2
00:43:59.585 | INFO     | Simulación 05  | INICIO  | t= 0.807s | concurrentes=1/2
00:43:59.585 | INFO     | Simulación 06  | INICIO  | t= 0.808s | concurrentes=2/2
00:43:59.990 | INFO     | Simulación 05  | FIN     | t= 1.212s | concurrentes=1/2
00:43:59.990 | INFO     | Simulación 06  | FIN     | t= 1.212s | concurrentes=0/2
00:43:59.990 | INFO     | Simulación 07  | INICIO  | t= 1.212s | concurrentes=1/2
00:43:59.990 | INFO     | Simulación 08  | INICIO  | t= 1.213s | concurrentes=2/2
00:44:00.404 | INFO     | Simulación 07  | FIN     | t= 1.627s | concurrentes=1/2
00:44:00.404 | INFO     | Simulación 08  | FIN     | t= 1.627s | concurrentes=0/2
00:44:00.404 | INFO     | Simulación 09  | INICIO  | t= 1.627s | concurrentes=1/2
```

```
00:44:00.404 | INFO     | Simulación 10  | INICIO  | t= 1.627s | concurrentes=2/2
00:44:00.808 | INFO     | Simulación 09  | FIN     | t= 2.030s | concurrentes=1/2
00:44:00.808 | INFO     | Simulación 10  | FIN     | t= 2.031s | concurrentes=0/2
00:44:00.808 | INFO     | Semáforo       | 10/10 OK en  2.031s
00:44:00.809 | INFO     | Ejercicio finalizado sin bloquear el event loop.
```

Diego Lalanda · Coderhouse AI Engineer